

Ch12: Bubble Sort

Slide 1: Understanding Bubble Sort

Bubble Sort is a sorting algorithm that repeatedly compares **two neighboring elements** in an array.

If the two elements are in the wrong order, the program **swaps them**.

Think of two people standing next to each other in a line. If the taller person is standing before the shorter person when the line should be shortest-to-tallest, they exchange positions. The program keeps doing this with neighboring elements until the array becomes sorted.

For ascending order:

smaller → larger

Consider:

5 3 8 2

The program starts from the beginning and compares neighbors:

5 and 3

Since:

$5 > 3$

they are in the wrong order for ascending sorting.

So they swap:

3 5 8 2

The program then moves to the next pair:

5 and 8

Since:

$5 < 8$

they are already in the correct order.

No swap is needed.

Then it compares:

8 and 2

Since:

$8 > 2$

they must be swapped:

3 5 2 8

Notice something important: **8 has moved all the way toward the end.**

This is where the name **Bubble Sort** comes from. Larger values gradually move toward the end of the array, like a value "bubbling" upward through repeated neighbor comparisons.

Key idea:

Bubble Sort repeatedly compares adjacent elements and swaps them when they are in the wrong order.

Slide 2: Comparing and Swapping Neighbors

Let's examine the first pass more carefully.

Starting array:

5 3 8 2

Comparison 1

Compare the first two elements:

5 3

For ascending order:

$5 > 3$

So swap them:

3 5 8 2

Comparison 2

Now move one position to the right.

Compare:

5 8

Check:

$5 > 8$

This is false.

So we do not swap.

3 5 8 2

Comparison 3

Move again.

Compare:

8 2

Check:

$8 > 2$

This is true.

Swap:

3 5 2 8

The entire pass looks like this:

Start

↓

5 3 8 2

↓

Compare 5 and 3

↓

3 5 8 2
↓
Compare 5 and 8
↓
3 5 8 2
↓
Compare 8 and 2
↓
3 5 2 8

The important observation is:

8
↓
3 5 2 8

The largest value has reached the final position.

Key idea:

During an ascending Bubble Sort pass, the largest unsorted value gradually moves toward the end.

Slide 3: Understanding a Bubble Sort Pass

A **pass** means moving through the array and comparing adjacent elements from left to right.

Starting array:

5 3 8 2

Let's trace one complete pass.

Step	Comparison	Result
Start		5 3 8 2
1	5 and 3	3 5 8 2
2	5 and 8	3 5 8 2
3	8 and 2	3 5 2 8

After one pass:

3 5 2 8

The array is **not completely sorted yet**.

We still have:

3 5 2

to organize.

So another pass is required.

Second Pass

Start:

3 5 2 8

Compare:

3 and 5

No swap.

3 5 2 8

Compare:

5 and 2

Swap:

3 2 5 8

Now the next comparison would involve the already-correct 8, so there is no need to disturb it.

Third Pass

3 2 5 8

Compare:

3 and 2

Swap:

2 3 5 8

Now the array is sorted:

2 3 5 8

What happened?

Pass 1 → 8 reaches the end

Pass 2 → 5 reaches its position

Pass 3 → 3 reaches its position

Visualize the sorted part growing from the right:

5 3 8 2

↓

3 5 2 | 8

↓

3 2 | 5 8

↓

2 | 3 5 8

The | separates the **unsorted portion** from the **sorted portion**.

Key idea:

Bubble Sort needs multiple passes. After each pass, one more large value reaches its correct position at the end.

Slide 4: Bubble Sort in C

The basic Bubble Sort structure uses **two loops**.

```
for (int i = 0; i < n - 1; i++)
{
    for (int j = 0; j < n - i - 1; j++)
    {
        if (arr[j] > arr[j + 1])
        {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

```
}
```

Let's understand what each part does.

Outer Loop

```
for (int i = 0; i < n - 1; i++)
```

The outer loop controls the **number of passes**.

For an array containing n elements, Bubble Sort can require up to:

$n - 1$ passes

Inner Loop

```
for (int j = 0; j < n - i - 1; j++)
```

The inner loop moves through the array and compares neighboring elements.

The important part is:

```
arr[j]  
arr[j + 1]
```

These represent two adjacent elements.

For example:

```
Index: 0  1  2  3  
      ↓  ↓  
Array: 5  3  8  2
```

At $j = 0$:

```
arr[j]    = 5  
arr[j + 1] = 3
```

So the program compares:

5 and 3

Then j increases:

```
Index: 0  1  2  3  
      ↓  ↓  
Array: 5  3  8  2
```

Now it compares:

3 and 8

And so on.

The Comparison

```
if (arr[j] > arr[j + 1])
```

This is the rule for **ascending order**.

If the left value is larger than the right value, they are in the wrong order.

So the program swaps them.

left > right

↓

Swap

The swap uses the temporary variable:

```
int temp = arr[j];  
arr[j] = arr[j + 1];  
arr[j + 1] = temp;
```

This safely exchanges the two neighboring values.

Why $n - i - 1$?

After each pass, one large value reaches the end.

For example:

```
Pass 1 →      8 is sorted  
Pass 2 →    5 8 are sorted  
Pass 3 →  3 5 8 are sorted
```

There is no need to compare elements that are already in the sorted section.

Therefore, the inner loop becomes shorter after each pass.

Key idea:

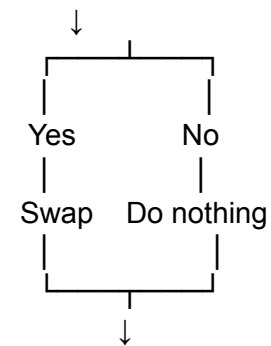
The outer loop creates passes, the inner loop compares neighbors, and the `if` condition decides whether a swap is needed.

Slide 5: The Main Idea of Bubble Sort

Bubble Sort can be understood as one repeated process:

Compare neighbors

↓
Are they in the wrong order?



Move to next pair

↓
Repeat the pass

↓
Start another pass

For ascending order:

```
if (arr[j] > arr[j + 1])
```

For descending order, the comparison changes:

```
if (arr[j] < arr[j + 1])
```

So the overall Bubble Sort structure can remain almost the same. The comparison determines the sorting direction.

Bubble Sort Mental Model

Array

↓
Compare adjacent elements

↓
Swap if wrong

↓
Continue across array
↓
Largest remaining value reaches the end
↓
Repeat
↓
Sorted array

For:

5 3 8 2

the values gradually move toward their correct positions:

5 3 8 2

↓

3 5 2 8

↓

3 2 5 8

↓

2 3 5 8

The important behavior is not simply "Bubble Sort uses two loops."

The real idea is:

Compare neighboring elements, swap when they are in the wrong order, and repeat until the array becomes sorted.

Core Takeaway

- **Adjacent elements** are compared.
- **Wrongly ordered neighbors** are swapped.
- **Multiple passes** are required.
- In ascending order, **larger values move toward the end**.
- The sorted portion grows after every pass.
- The outer loop controls **passes**.
- The inner loop controls **neighbor comparisons**.
- The comparison condition determines **ascending or descending order**.